Feedback

Workshop 5

Workshop 5 feedback

- The fifth workshop looked at floating point accuracy and exceptions
  - The first section was about measuring the accuracy of a `float` and `double`
  - The second section was about comparing numerical errors and round-off errors
  - The third section was about trapping and fixing a floating point exception

Workshop 5

Question 1: How many digits of accuracy does a float provide?

Question 2: How many digits of accuracy does a double provide and is this consistent with IEEE754 double precision?

```
> ./float
 8 digits accuracy in calculations
 8 digits accuracy in storage
> ./double
16 digits accuracy in calculations
16 digits accuracy in storage
>
```

A `float` provides 8 digits of accuracy

A `double` provides 16 digits of accuracy

The machine epsilon is $2^{-52} \approx 10^{-16}$ for double precision

Workshop 5

$$\sin(x) = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \cdots$$

Question 3: How many terms are required in the power series to compute $sin(0.5)$ to within the round-off error of the reduced precision floating point format?

```
> ./sine 0.5
x**1      5.000E-01
1!        1.000E+00
x**1/1! 5.000E-01

  total    5.000E-01

x**3      1.250E-01
3!        6.000E+00
x**3/3! 2.083E-02

  total    4.792E-01

x**5      3.125E-02
5!        1.200E+02
x**5/5! 2.604E-04

  total    4.794E-01

x**7      7.812E-03
7!        5.040E+03
x**7/7! 1.550E-06

4 term computed
sin(5.000E-01)=
  4.794E-01
Actual sin(0.5)=0.479426
```

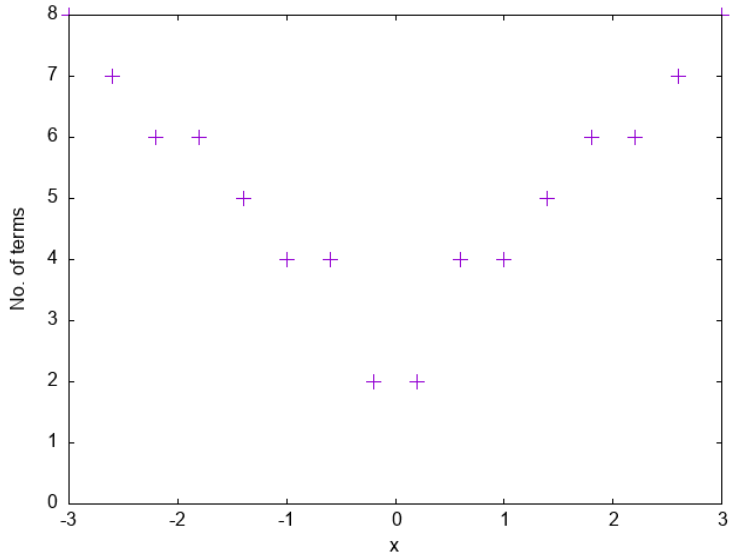
3 terms are required:

This is the last term that makes a difference to the series at this precision

The 4th term is computed but is too small to contribute to the sum

Workshop 5

Question 4: How does the number of terms required for convergence vary with the argument of the sine function (x)?



The power series is a more accurate approximation as x gets closer to zero so we need fewer terms to converge to the floating point precision

Workshop 5

Question 5: Why is a `NaN` being generated and how would you modify the program to stop this happening?

```
> ./sinc
NaN generated: i=500, x=0.000000
```

```
/* Calculate values of x and y writing them out as we go */
for (i=0; i<N; i++){
  x = range * PI * ( (float) (i-N/2) ) / ( (float) (N/2) );
  y = sin(x) / x;

  /* Report when a NaN is generated */
  if (isnan(y)){
    printf("NaN generated: i=%d, x=%f\n", i, x);
  }

  /* Write values of x and y to the output file */
  fprintf(outfile, "%f, %f\n", x, y);
}
```

When `x=0.0` this line computes `0.0/0.0` which is an invalid operation

Workshop 5

What should we do if `x=0.0`? Can we trap this condition and handle it as a special case?

$$\sin(x) = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \cdots$$

The number of terms required decreases as x gets smaller and $\sin(x) \approx x$ is a good approximation near $x = 0$

$$\frac{\sin(x)}{x} \approx \frac{x}{x} = 1$$

If $x = 0$ then $\mathrm{sinc}(x) = 1$ (this isn't a formal proof but gives a plausible explanation)

Workshop 5

```
/* Calculate values of x and y writing them out as we go */
for (i=0; i<N; i++){
  x = range * M_PI * ( (float) (i-N/2) ) / ( (float) (N/2) );
  if (x==0.0){
    y=1.0;
  } else {
    y = sin(x) / x;
  }

  /* Report when a NaN is generated */
  if (isnan(y)){
    printf("NaN generated: %d, %f\n", i, x);
  }

  /* Write values of theta and I to the output file */
  fprintf(outfile, "%f, %f\n", x, y);
}
```

Trap `x=0.0` to avoid generating an exception in the first place

Workshop 6

- Workshop 6 will use Isca (the university HPC system) instead of the Lovelace lab PCs
- Isca is assessed by connecting to a log in node using SSH (see Unit 1.4 “The cluster architecture”)
- The log in process is similar to accessing the Lovelace lab PCs
- Before the workshop please check that you can log on to Isca as described on the following slides

Before workshop 6: step 1

- Log in to Isca:

Your computer needs to be on the University network or VPN to connect to Isca

```
ssh abc123@login.isca.ex.ac.uk
```

where you should replace `abc123` with your university username and enter your university password when prompted

- This should log you in and display the welcome message

```
********************************************************************************
*                 UNIVERSITY OF EXETER ISCA HPC SERVICE                        *
*                                                                              *
* This computer system is operated on behalf of the University of Exeter.      *
* Only authorised users are entitled to connect and/or login to this           *
* computer system. If you are not sure whether you are authorised, then        *
* you are not and should DISCONNECT IMMEDIATELY.                               *
*                                                                              *
* For user documentation please visit                                          *
*    https://universityofexeteruk.sharepoint.com/sites/ExeterARC               *
* Email isca-support@exeter.ac.uk with any direct support issues               *
*                                                                              *
*              CURRENT TIMEOUT FOR INACTIVITY IS 24 HOURS                      *
*                                                                              *
********************************************************************************
```

Before workshop 6: step 2

- Check you can see the directory which will contain the workshop materials:

```
cat /lustre/projects/Research_Project-IscaTraining/HPC/README
```

- You should see the following output:

```
This directory contains material for the High Performance Computing workshops.
```

Before workshop 6: step 3

- Log out by typing the command `exit`
- This should log you out from Isca:

```
logout
Connection to login.isca.ex.ac.uk closed.
```

- If there is a problem with any of these steps please email TA with a description of the problem so it can be resolved before the workshops start